KALASALINGAM ACADEMY OF RESEARCH AND EDUCATION

AI-Powered Interview Coach: A Resume-Tailored Simulation Platform

Project Documentation

Project ID: PRJ011

Team ID: T011

Team GitHub Repository: github.com/DoddiSurendra55555/gen-ai-project

Student Name	Registration Number	Individual GitHub Link
G. Naga Mithra Reddy	99230040226	https://github.com/gangireddynagamithareddy-create/Gen-AI
Vinukonda Reshma	99230040808	https://github.com/reshma781/Gen-AI-task
C. Lahari	99230040920	https://github.com/Lahari200663/Gen_AI-files
D. Surendra	99230040525	https://github.com/DoddiSurendra55555
P. Kavitha	99230040718	https://github.com/kavithasrinupathipati-create/Gen-AI

Project Trainer / Guide

Mr. SHIRWIN PRINCE I

Table of Contents

1. Introduction	Section 1
2. Problem Statement and Objectives	Section 2
3. System Architecture	Section 3
3.1 Overall Application Architecture	Section 3
3.2 Resume Processing	Section 3
3.3 Hybrid RAG Architecture	Section 3
3.4 Document Chunking and Embedding Generation	Section 3
3.5 Hybrid Knowledge Store	Section 3
3.6 Semantic Retrieval Using Cosine Similarity	Section 3
3.7 RAG-Based Interview Question Generation	Section 3
3.8 RAG-Based Answer Evaluation	Section 3
3.9 Follow-Up Questions and Interview Results	Section 3
4. Implementation Details and Technology Stack	Section 4
5. Results and Testing	Section 5
6. Privacy and Data Handling	Section 6
7. Conclusion and Future Scope	Section 7

1. Introduction

Traditional interview preparation often relies on general question banks and static interview materials. These questions may not reflect the candidate's actual skills, projects, educational background, or professional experience.

Candidates often face difficulties when explaining their resume projects, demonstrating technical knowledge, and answering questions related to their individual experience. They also need proper feedback to identify their strengths and improve their weaknesses.

To address these challenges, this project introduces an **AI-Powered Interview Coach**, a web-based application that provides personalized interview preparation using Artificial Intelligence.

The application is developed using Python and Streamlit and integrates Google's Gemini AI models for resume analysis, interview question generation, answer evaluation, and feedback generation.

An important feature of this project is the integration of **Hybrid Retrieval-Augmented Generation (Hybrid RAG)**.

Hybrid RAG combines two knowledge sources:

1. Candidate-specific information extracted from the uploaded resume.
2. A general interview knowledge base containing technical, programming, software engineering, behavioral, and HR interview concepts.

The system retrieves relevant information from both sources and provides the retrieved context to Google Gemini. This helps generate personalized interview questions and evaluate candidate answers using both individual resume information and general interview knowledge.

The application also supports browser-based voice interaction, adaptive follow-up questions, performance analysis, and downloadable interview reports.

2. Problem Statement and Objectives

2.1 Problem Statement

Many candidates do not have access to affordable and personalized interview coaching.

Traditional interview preparation platforms often provide generic questions that do not consider the candidate's actual resume, skills, projects, and experience.

Candidates may also struggle to identify weaknesses in their answers because they do not receive detailed and structured feedback.

Therefore, there is a need for an intelligent interview preparation platform that can analyse a candidate's resume, generate personalized questions, evaluate answers, and provide useful recommendations.

2.2 Objectives

The main objectives of the project are:

1. To develop an AI-powered interview preparation application.
2. To extract text from uploaded PDF resumes.

3. To analyse candidate skills, education, projects, and experience using Google Gemini.
4. To generate personalized technical, HR, and behavioral interview questions.
5. To implement Hybrid Retrieval-Augmented Generation using resume information and general interview knowledge.
6. To generate text embeddings for semantic document retrieval.
7. To retrieve relevant information using cosine similarity.
8. To evaluate interview answers based on relevance, accuracy, clarity, and completeness.
9. To provide scores, strengths, improvement suggestions, and example stronger answers.
10. To support voice-based interview interaction through browser speech technologies.
11. To generate adaptive follow-up questions based on previous answers.
12. To provide a final interview performance report.

3. System Architecture

3.1 Overall Application Architecture

The application follows a Python-based Streamlit architecture. Streamlit is used to develop the user interface and manage the multipage application workflow.

The application consists of four main pages:

1. Resume Analysis
2. Interview Setup
3. Mock Interview
4. Interview Results

The application workflow is as follows:

1. The candidate uploads a PDF resume.
2. The application extracts the resume text.
3. Google Gemini analyses the extracted resume information.
4. The candidate selects the job role, interview type, and difficulty level.
5. Hybrid RAG retrieves relevant information from the resume and general knowledge base.
6. Gemini generates personalized interview questions.
7. The candidate answers questions using voice or text.
8. Hybrid RAG retrieves relevant context for answer evaluation.
9. Gemini evaluates the candidate's answers.
10. Follow-up questions are generated when required.
11. The application displays the final interview report and performance analysis.

3.2 Resume Processing

The resume processing module allows candidates to upload their resumes in PDF format. The application uses PyMuPDF to extract text from the uploaded PDF document. The extracted text is passed to Google Gemini for analysis.

The AI analyses information such as:

- Educational qualifications.
- Technical skills.
- Programming languages.
- Projects.
- Work experience.
- Certifications.
- Other relevant resume information.

The extracted resume text is also used as the candidate-specific knowledge source in the Hybrid RAG architecture. This allows the system to generate questions based on the candidate's actual resume content.

3.3 Hybrid RAG Architecture

What is Hybrid RAG?

Retrieval-Augmented Generation (RAG) is a technique that improves Large Language Model responses by retrieving relevant information from an external knowledge source and supplying it as context to the model.

Hybrid RAG combines multiple knowledge sources to provide more relevant information to the language model. In this project, Hybrid RAG combines:

Source 1: Candidate Resume. The uploaded resume contains candidate-specific information, including education, technical skills, projects, and experience.

Source 2: General Interview Knowledge Base. The general knowledge base contains information about technical concepts, programming, software development, behavioral interviews, and HR interviews.

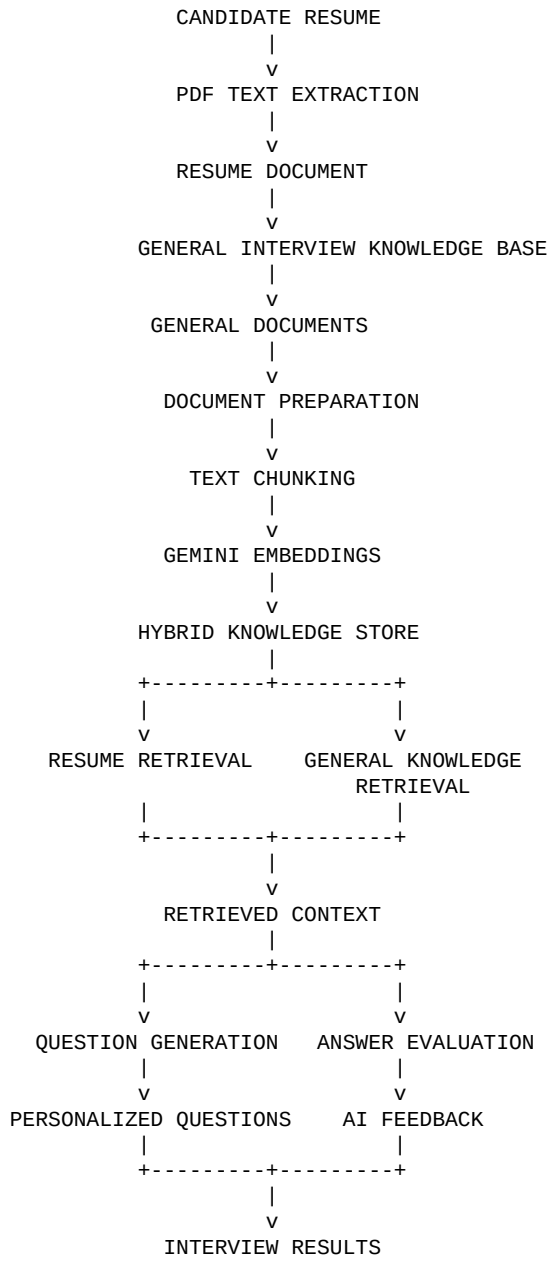
By combining these sources, the system can understand both the candidate's background and the general knowledge expected in an interview.

Hybrid RAG Workflow

The Hybrid RAG workflow consists of the following stages:

1. Resume text extraction.
2. General knowledge document preparation.
3. Document chunking.
4. Embedding generation.
5. Hybrid knowledge store construction.
6. Semantic retrieval.
7. Context combination.
8. AI-based question generation and answer evaluation.

Hybrid RAG Architecture Diagram



3.4 Document Chunking and Embedding Generation

After preparing the documents, the RAG engine divides them into smaller text sections called chunks. Chunking helps the retrieval system identify specific relevant information instead of processing entire documents for every query.

Each chunk retains its source information, identifying whether it belongs to the candidate's resume or the general knowledge base.

The project uses Google's **gemini-embedding-001** model to generate embeddings. Embeddings are numerical vector representations of text that capture semantic meaning. The project uses a configured embedding dimension of 768. The generated embeddings are used for semantic similarity calculations during the retrieval process.

3.5 Hybrid Knowledge Store

The hybrid knowledge store contains the prepared resume and general knowledge chunks along with their embeddings and source information.

The system maintains the knowledge store in Streamlit session state. The store is reused during the interview session to avoid unnecessary rebuilding.

A hash of the resume text is used to identify whether the resume has changed. If the resume changes or the knowledge store is unavailable, the system rebuilds the store.

Candidate-specific embeddings are maintained in session memory rather than being stored in a permanent vector database.

3.6 Semantic Retrieval Using Cosine Similarity

When the system generates an interview question or evaluates an answer, it creates a retrieval query. The query is converted into an embedding using the Gemini embedding model. The system retrieves relevant chunks separately from the two knowledge sources.

Resume Retrieval: This process identifies resume sections relevant to the interview question and candidate answer.

General Knowledge Retrieval: This process identifies general interview concepts relevant to the question and expected answer.

The project uses cosine similarity to measure the similarity between the query embedding and document embeddings. The most relevant chunks are selected from both sources.

In the current implementation, the answer evaluator retrieves up to three resume chunks and three general knowledge chunks. The retrieved information is combined into a single context and passed to Google Gemini. This allows Gemini to generate responses based on both candidate-specific and general interview knowledge.

3.7 RAG-Based Interview Question Generation

The Interview Setup page integrates the Hybrid RAG question-generation module. The module receives the following information:

- Candidate resume text.
- Resume analysis.
- Selected job role.
- Interview type.
- Difficulty level.
- Number of questions.

The Hybrid RAG system prepares the knowledge store and retrieves relevant information from the candidate's resume and general interview knowledge base. The retrieved context is passed to Google Gemini along with the interview configuration.

Gemini generates personalized interview questions based on the retrieved information. The generated questions include details such as:

- Interview question.

- Question category.
- Expected topics.

The generated questions are stored in the Streamlit session state and used by the Mock Interview page. This makes the interview more personalized than using only a static question bank.

3.8 RAG-Based Answer Evaluation

The Mock Interview page integrates the Hybrid RAG answer evaluator. When a candidate submits an answer, the system uses the active question, candidate answer, resume text, resume analysis, and expected topics to prepare a retrieval query.

The Hybrid RAG evaluator retrieves relevant information from both the resume and general interview knowledge base. The retrieved context is supplied to Google Gemini. Gemini evaluates the answer using the following criteria:

1. Relevance to the interview question.
2. Technical or conceptual accuracy.
3. Clarity of explanation.
4. Completeness of the answer.
5. Consistency with the candidate's stated background.

The evaluator is instructed not to invent candidate achievements or assume that general knowledge represents the candidate's personal experience. The evaluation produces a structured response containing:

- Score between 0 and 100.
- Detailed feedback.
- Strengths.
- Areas for improvement.
- Example of a stronger answer.

The same Hybrid RAG evaluation function is used for both main interview answers and follow-up answers.

3.9 Follow-Up Questions and Interview Results

The application supports adaptive follow-up questions. Based on the candidate's previous answer and its evaluation, Gemini can generate additional questions to explore the candidate's understanding.

The application supports up to two follow-up questions for each main interview question. Follow-up question generation uses the existing Gemini-based question-generation function, while the evaluation of follow-up answers uses Hybrid RAG.

After the interview is completed, the Interview Results page displays:

- Individual answer scores.
- Overall interview performance.
- Feedback for each answer.
- Strengths.
- Areas for improvement.
- Example stronger answers.

- Follow-up evaluation details.

Plotly is used to visualize interview performance. The final results can also be downloaded in JSON format. The final report generation continues to use the existing Gemini-based reporting function.

4. Implementation Details and Technology Stack

The project uses the following technologies.

Component	Technology	Purpose
Programming Language	Python	Application development
Frontend Framework	Streamlit	User interface and navigation
Generative AI	Google Gemini API	Resume analysis, question generation, evaluation
Embedding Model	gemini-embedding-001	Text embedding generation
PDF Processing	PyMuPDF	Resume text extraction
Retrieval	Cosine Similarity	Semantic document retrieval
Data Handling	Python	Document and knowledge-store management
Visualization	Plotly	Interview performance charts
Voice Recognition	Browser Web Speech API	Speech-to-text
Voice Output	Browser Text-to-Speech	Reading questions aloud
Data Export	JSON	Downloadable interview results
Testing	Pytest	Automated testing

4.1 Main Project Modules

Module	Description
app.py	The main entry point of the Streamlit application.
pages/1_Resume_Analysis.py	Handles resume uploading, PDF text extraction, and AI-based resume analysis.
pages/2_Interview_Setup.py	Collects interview settings and generates personalized questions using Hybrid RAG.
pages/3_Mock_Interview.py	Manages the mock interview, voice and text answers, Hybrid RAG evaluation, and follow-up question workflow.
pages/4_Interview_Results.py	Displays interview scores, feedback, performance charts, and downloadable results.
utils/ai_engine.py	Gemini-based AI functions for resume analysis, question generation, answer evaluation, follow-up question generation, and final report generation.
utils/resume_parser.py	Handles PDF resume extraction and validation.
utils/rag_engine.py	Document chunking, embedding generation, knowledge-store construction, cosine similarity, and relevant chunk retrieval.
utils/hybrid_rag.py	Combines candidate resume documents and general interview knowledge and performs separate retrieval from both sources.
utils/rag_question_generator.py	Uses retrieved Hybrid RAG context to generate personalized interview questions.
utils/rag_evaluator.py	Uses retrieved Hybrid RAG context to evaluate main interview answers and follow-up answers.
knowledge_base/interview_knowledge.py	The general interview knowledge base with 15 interview-related topics.
utils/browser_storage.py	Supports browser-side session storage functionality.
utils/browser_voice.py and browser_voice_component/	Support browser-based voice interaction.
utils/voice_engine.py	Supports voice-related application functionality.

5. Results and Testing

5.1 RAG Module Testing

The RAG engine was tested for successful importing, document chunking, and cosine similarity calculation.

The general interview knowledge base was tested and confirmed to contain 15 interview-related topics.

The Hybrid RAG module was tested to confirm that it could combine resume documents and general knowledge documents and build the hybrid knowledge store.

The RAG question-generation module was tested by generating personalized interview questions and checking the retrieved knowledge summary.

The RAG evaluator module was tested for successful importing and integration with the Mock Interview workflow.

5.2 Application Workflow Testing

The integrated application was tested through the following workflow:

1. Uploading a candidate resume.
2. Extracting and analysing resume information.
3. Configuring the interview.
4. Generating questions using Hybrid RAG.
5. Submitting answers to the interview questions.
6. Evaluating answers using retrieved resume and general knowledge.
7. Generating follow-up questions.
8. Evaluating follow-up answers using Hybrid RAG.
9. Viewing the final interview report.

The developer confirmed that the integrated workflow operated successfully.

5.3 Automated Testing

The project includes automated tests for selected utility modules. The tests can be executed using:

```
python -m pytest tests -v
```

Python syntax compilation for the RAG modules can be checked using:

```
python -m py_compile utils/rag_engine.py utils/hybrid_rag.py
               utils/rag_question_generator.py utils/rag_evaluator.py
```

These tests help identify syntax errors and verify selected application functions.

6. Privacy and Data Handling

The application processes candidate resumes and interview answers to provide personalized interview preparation.

The resume-specific knowledge store is maintained in Streamlit session state. The current Hybrid RAG implementation does not use a permanent vector database to store candidate-specific embeddings.

Resume content, submitted answers, and retrieved context are sent to Google Gemini for AI processing.

The application does not provide permanent resume storage or a user authentication system by default.

API keys are stored locally using environment configuration or through deployment secrets. The **.env** file must not be uploaded to GitHub or shared publicly.

7. Conclusion and Future Scope

7.1 Conclusion

The AI-Powered Interview Coach provides an intelligent and personalized interview preparation platform using resume analysis, generative AI, browser-based voice interaction, adaptive follow-up questions, and structured feedback.

The integration of Hybrid Retrieval-Augmented Generation improves the context supplied to Google Gemini during interview question generation and answer evaluation.

By combining candidate-specific resume information with a general interview knowledge base, the system can generate more relevant interview questions and provide context-aware evaluations.

The application uses Gemini embeddings and cosine similarity to retrieve relevant information from both knowledge sources. The retrieved information is supplied to Gemini to support personalized question generation and answer evaluation.

The implemented Hybrid RAG workflow has been tested successfully as part of the application.

7.2 Future Scope

The following improvements can be considered in future versions:

1. Expanding the general interview knowledge base with more technical and professional topics.
2. Improving document chunking and semantic retrieval strategies.
3. Evaluating retrieval quality using larger and more diverse test datasets.
4. Supporting additional resume document formats.
5. Introducing user authentication and secure interview history management.
6. Adding detailed interview performance analytics.
7. Supporting real-time streaming of AI feedback.
8. Improving browser-based voice interaction.
9. Adding optional analysis of speaking pace and communication delivery.
10. Exploring a dedicated vector database if the system requires larger-scale knowledge storage.

The current project provides a foundation for further development of intelligent, personalized interview preparation systems.